

Revisiting PUF-based User Authentication in WSNs: Flaws and Remedies (Appendix File)

Meijia Xu, Bin Xue, Peizhen Hong, and Ding Wang

Abstract—This appendix shows the formal security analysis of our PUF-based MFA scheme under the ROM model.

Index Terms—Appendix file.

APPENDIX A: FORMAL ANALYSIS OF SECURITY UNDER THE ROM MODEL

Before the security analysis, we introduce the computational assumption required for the formal security proof: **Elliptic Curve Computational Diffie-Hellman (ECCDH) problem.**: A (t, ϵ) -ECCDH attacker in $\mathcal{G}$ is a PPT machine Δ running in time t such that

$$\begin{aligned} \text{Adv}_{g, \mathcal{G}}^{\text{ECCDH}}(\Delta) &= \Pr_{x, y} [\Delta(xP, yP) = xyP] \\ \text{Adv}_{g, \mathcal{G}}^{\text{ECCDH}}(t) &= \max_{\Delta} \left\{ \text{Adv}_{g, \mathcal{G}}^{\text{ECCDH}}(\Delta) \right\} \end{aligned}$$

the probability is taken over the random values x and y . The ECCDH-Assumption states $\text{Adv}_{g, \mathcal{G}}^{\text{ECCDH}}(t) \leq \epsilon$, where ϵ is a negligible value.

Proof. For simplicity, this proof omits considerations of forward secrecy. We methodically establish a series of games, beginning with the actual attack scenario, Game G_0 , and culminating with Game G_8 .

For each iteration of the game, labeled G_n ($n = 0, 1, \dots, 8$), we specify the following events:

Succ_n: The adversary $\mathcal{A}$ successfully guesses the bit b in Test-query correctly.

AskPara_n: The adversary $\mathcal{A}$ correctly computes the parameter K_i by asking a hash query $H(\cdot)$ on $(ID_i || X_{GWN} || T_i)$ or $(ID_i || PW_i || \alpha_i || b)$.

AskAuth_n: $\mathcal{A}$ correctly computes the parameter K_i and asks a hash $H(\cdot)$ on $(ID_i || X_1 || X_2 || K_i || C_1 || a_2 || SID_j)$ or $(a_2 || X_1 || X_2 || X_3 || K_i || SID_j || N_1)$.

AskH_n: On the basis of the above hash-queries, the adversary $\mathcal{A}$ successfully asks a hash query on $(X_1 || X_3 || X_4 || a_2 || SID_j)$ to compute the session key SK .

This work was supported in part by Natural Science Foundation of Tianjin, China under Grant 25JCJQJC00230 and by supported by the Fundamental Research Funds for the Central Universities, Nankai University under Grant 63263258.

Meijia Xu and Ding Wang are with the College of Cryptology and Cyber Science, Nankai University, Tianjin 300350, China, and Key Laboratory of Data and Intelligent System Security (Ministry of Education), Nankai University, Tianjin 300350, China, and also with Tianjin Key Laboratory of Network and Data Security Technology, Nankai University, Tianjin 300350, China. Bin Xue and Peizhen Hong are with the College of Electronic Information and Optical Engineering, Nankai University, Tianjin 300350, China, and also with Tianjin Key Laboratory of Optoelectronic Sensor and Sensing Network Technology, Nankai University, Tianjin 300350, China. (E-mail: xumeijia@mail.nankai.edu.cn; binxue@mail.nankai.edu.cn; hongpeizhen@nankai.edu.cn; wangding@nankai.edu.cn;).

Game G_0 . This game corresponds to a real attack under the random oracle model, thus there are

$$\text{Adv}_{\mathcal{P}}^{ake}(A) = 2 \Pr[\text{Succ}_0] - 1 \quad (1)$$

Game G_1 . In this game, we simulate the hash-query $H(\cdot)$ and $H'(\cdot)$ as they will appear in the later Game G_6 . The system maintains two separate hash lists: $\Lambda_{\mathcal{H}}$ for recording hash queries and $\Lambda_{\mathcal{A}}$ for tracking the hash queries initiated by the adversary $\mathcal{A}$. When a hash query $H(x)$ is received, with x as the input, the oracle first checks if $\Lambda_{\mathcal{H}}$ already includes a record (x, y) . If this record exists, the response is y . If not, the oracle generates a random number y from the set $0, 1^l$, returns this as the output, and adds the record (x, y) to $\Lambda_{\mathcal{H}}$. The $\Lambda_{\mathcal{A}}$ list specifically captures the hash query records requested by the adversary. The simulation of other queries remains consistent with what the real players would do. Consequently, this game mirrors a real attack scenario.

$$|\Pr[\text{Succ}_1] - \Pr[\text{Succ}_0]| = 0 \quad (2)$$

Game G_2 . In this game, the behavior of all oracles is consistent with that observed in Game G_1 . To aid in further analysis, it is necessary to eliminate potential (unlikely) collisions. The collisions include the following two types:

-The collisions on the output of hash queries.

-The collisions on the message $\{\{C_1, C_2, TID_i, X_1\}, \{G_1, G_2, X_1\}, \{N_1, N_2, X_3\}, \{N_1, G_3, X_3\}\}$.

According to the birthday paradox principle, we have

$$|\Pr[\text{Succ}_2] - \Pr[\text{Succ}_1]| \leq \frac{(q_{\text{send}} + q_{\text{exe}})^2}{2p} + \frac{q_h^2}{2^{l+1}} \quad (3)$$

Game G_3 . In this game, we consider the situation that if the adversary is lucky to guess the correct verifier directly $\{C_2, G_2, N_2, G_3\}$ in the protocol (without asking the hash query $H(\cdot)$), then we have

$$|\Pr[\text{Succ}_3] - \Pr[\text{Succ}_2]| \leq \frac{q_{\text{send}}}{2^l} \quad (4)$$

Game G_4 . In this game, we abort the game if the adversary is lucky to directly guess the K_i correctly (without asking the hash query $H(\cdot)$). Then we have

$$|\Pr[\text{Succ}_4] - \Pr[\text{Succ}_3]| \leq \frac{q_{\text{send}}}{2^l} \quad (5)$$

Game G₅. In this game, we abort the game if the $\mathcal{A}$ can compute the correct K_i by asking the corresponding hash query. In the case where the $AskPara_5$ event does not occur, we cannot distinguish G_4 and G_5 . Then we have

$$|\Pr[\text{Succ}_5] - \Pr[\text{Succ}_4]| \leq \Pr[AskPara_5] \quad (6)$$

Since the case of directly guessing K_i 's value has been ruled out before, the $AskPara_5$ event can be divided into the following two cases:

- $\mathcal{A}$ asks a **Corrupt**($U, 1$) query to compute K_i .
- $\mathcal{A}$ asks a **Corrupt**($U, 2$) query to compute K_i .

We define these two events as $AskPara_5 \text{ with } Corrupt_1$ and $AskPara_5 \text{ with } Corrupt_2$ respectively. We first consider the probability of the event $AskPara_5 \text{ with } Corrupt_1$. The adversary uses a $Corrupt(U, 1)$ query to obtain the user's password PW_U . Since $\mathcal{A}$ does not know other parameters, the probability of this event is the same as direct guessing. Then we have

$$\Pr[AskPara_5 \text{ with } Corrupt_1] \leq \frac{q_{\text{send}}}{2^l} \quad (7)$$

With $Corrupt(U, 2)$, $\mathcal{A}$ obtains the stored parameters in AE_i but cannot directly read raw PUF responses. $\mathcal{A}$ can only invoke the PUF through the rate-limited local verification interface. Therefore, the probability of recovering K_i is bounded by the success probability of online password guessing under the login-attempt limit.

With a $Corrupt(U, 2)$ query, $\mathcal{A}$ obtains the parameters stored in the auxiliary device AE_i , but cannot directly read the raw PUF response. The adversary can only invoke the PUF through the local verification interface, where failed trials are rate-limited by the counter. Let ρ denote the maximum number of failed trials allowed in each reset period and let η denote the number of reset periods within the attack window. Then the total number of password trials is bounded by $q_{\text{try}} \leq \rho\eta$. Therefore,

$$\begin{aligned} \Pr[AskPara_5 \text{ with } Corrupt_2] &\leq \frac{1}{2} C' \cdot q_{\text{try}}^{s'} \\ &\leq \frac{1}{2} C' \cdot \min\{q_{\text{send}}, \rho\eta\}^{s'} \leq \frac{1}{2} c' \cdot q_{\text{send}}^{s'}. \end{aligned} \quad (8)$$

In our scheme, $\rho = 5$ and the reset period is 24 hours.

According to the above equations, we have

$$|\Pr[\text{Succ}_5] - \Pr[\text{Succ}_4]| \leq \frac{1}{2} c' \cdot q_{\text{send}}^{s'} + \frac{q_{\text{send}}}{2^l} \quad (9)$$

Game G₆. We abort the game if $\mathcal{A}$ can compute the correct C_2, G_3 by asking the corresponding hash query and impersonating a user or the GWN. The G_5 and G_6 are indistinguishable unless event $AskAuth_6$ occurs, then we have

$$|\Pr[\text{Succ}_6] - \Pr[\text{Succ}_5]| \leq \Pr[AskAuth_6] \quad (10)$$

$$\Pr[AskPara_6] - \Pr[AskPara_5] \leq \Pr[AskAuth_6] \quad (11)$$

Game G₇. In this game, we use private hash $H'(\cdot)$ to replace the hash $H(\cdot)$ in the protocol. The G_6 and G_7 are indistinguishable unless event $AskH_7$ occurs, then we have

$$|\Pr[\text{Succ}_7] - \Pr[\text{Succ}_6]| \leq \Pr[AskH_7] \quad (12)$$

$$|\Pr[AskAuth_7] - \Pr[AskAuth_6]| \leq \Pr[AskH_7] \quad (13)$$

Besides, since $\mathcal{A}$ cannot use the private hash $H'(\cdot)$, then

$$\Pr[\text{Succ}_7] = \frac{1}{2} \quad (14)$$

Game G₈. In this game, we use the random self-reducibility of the Computational Diffie-Hellman problem to simulate the executions. Since the G_8 and G_7 are indistinguishable, then

$$\Pr[AskH_8] = \Pr[AskH_7] \quad (15)$$

The event $AskH_8$ occurs means that $\mathcal{A}$ had queried the random oracles $H(\cdot)$ on $H(X_1 || X_3 || ECCDH(X_1, X_3) || a_2 || SID_j)$. Then we have

$$|\Pr[AskH_8]| \leq q_h \cdot \text{Adv}_{\mathcal{P}}^{\text{ECCDH}}(t') \quad (16)$$

where $t' \leq t + (q_{\text{send}} + q_{\text{exe}} + 1) \cdot T_m$.

Firstly, from the Eqs.(1)-(5), we have

$$|\Pr[\text{Succ}_4] - \Pr[\text{Succ}_0]| \leq \frac{(q_{\text{send}} + q_{\text{exe}})^2}{2p} + \frac{q_h^2}{2^{l+1}} + \frac{2q_{\text{send}}}{2^l}$$

Then, from Eqs.(6)-(12), we get

$$|\Pr[\text{Succ}_7] - \Pr[\text{Succ}_4]| \leq$$

$$\Pr[AskPara_5] + \Pr[AskAuth_6] + \Pr[AskH_7]$$

Lastly, from Eqs.(13)-(16), we can get

$$\text{Adv}_{\mathcal{P}}^{\text{ake}}(\mathcal{A}) = 2\Pr[\text{Succ}_7] - 1 + 2(\Pr[\text{Succ}_0] - \Pr[\text{Succ}_7]).$$

Then, we have

$$\begin{aligned} \text{Adv}_{\mathcal{P}}^{\text{ake}}(\mathcal{A}) &\leq C' \cdot q_{\text{send}}^{s'} + 6q_h \text{Adv}_{\mathcal{P}}^{\text{ECCDH}}(t') \\ &\quad + \frac{q_h^2 + 6q_{\text{send}}}{2^l} + \frac{(q_{\text{send}} + q_{\text{exe}})^2}{p} \end{aligned}$$

We prove that the adversary's advantage is gradually decreasing to zero via a series of hybrid games. Thus, the adversary cannot break the semantic security of the protocol. We formally prove the security of our protocol.